



Welcome to CSC 365

Internet Programming

Dr. Yifan Zhang

React key Prop

The `key` prop is used by React to uniquely identify elements within lists to speed up rendering.

- Always use a unique key for the parent-most element rendered in a list.
- This key needs to be unique among siblings.
- This key should usually not be the index of the item(e.g. what if the order changes?)

React Lists

Render lists with `Array.map()`.

```
16 function BasicList() {
17   const fruits = ["Apple", "Banana", "Cherry", "Dragonfruit"];
18
19   return (
20     <section>
21       <h2>Basic list (strings)</h2>
22       <ul>
23         {fruits.map((fruit) => (
24           // Fruit name is unique here, so it's okay as a key for this *static* list.
25           <li key={fruit}>{fruit}</li>
26         ))}
27       </ul>
28     </section>
29   );
30 }
```

React Lists

If you omit key, React warns and internally falls back to index matching. Or if you do this:

```
{fruits.map((fruit, i) => (  
  <li key={i}>{fruit}</li>           // ✗ using index as key  
))}
```

and you remove "Banana", the indices shift:

```
0: Apple    -> stays 0  
1: Banana   -> removed  
2: Cherry   -> becomes index 1  
3: Dragonfruit -> becomes index 2
```

Responsive Design

We use `react-bootstrap`.

Install using the following command:

```
npm install react-bootstrap bootstrap
```

Be sure to load Bootstrap's CSS once (e.g., in `main.jsx` or `index.js`):

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Important takeaways...

- Use `Container`, `Row`, and `Col` components.
- `xs`, `sm`, `md`, `lg`, `xl`, and `xxl` are props.

Responsive Design

This is how we wrote Bootstrap in Vanilla JS...

```
<div class="container">  
  <div class="row">  
    <div class="col-12 col-md-6 col-lg-3"></div>  
    <div class="col-12 col-md-6 col-lg-3"></div>  
    <div class="col-12 col-md-6 col-lg-3"></div>  
    <div class="col-12 col-md-6 col-lg-3"></div>  
  </div>  
</div>
```

Responsive Design

...this is how we will in React!

```
<Container>  
  <Row>  
    <Col xs={12} md={6} lg={3}></Col>  
    <Col xs={12} md={6} lg={3}></Col>  
    <Col xs={12} md={6} lg={3}></Col>  
    <Col xs={12} md={6} lg={3}></Col>  
  </Row>  
</Container>
```

Pagination

Also available in react-bootstrap.

Useful for handling large sums of data.

```
{/* Display items here. */}
```

```
<Pagination>
```

```
  <Pagination.Item active={false}>1</Pagination.Item>
```

```
  <Pagination.Item active={false}>2</Pagination.Item>
```

```
  <Pagination.Item active={false}>3</Pagination.Item>
```

```
  <Pagination.Item active={false}>4</Pagination.Item>
```

```
</Pagination>
```


Pagination

Use a state variable to track which page is active.

```
function SomeBigData() {  
  const [page, setPage] = useState(1)  
  return <div>  
    {/* Display some data here! */}  
    <Pagination>  
      <Pagination.Item active={page === 1} onClick={() => setPage(1)}>1</Pagination.Item>  
      <Pagination.Item active={page === 2} onClick={() => setPage(2)}>2</Pagination.Item>  
      <Pagination.Item active={page === 3} onClick={() => setPage(3)}>3</Pagination.Item>  
      <Pagination.Item active={page === 4} onClick={() => setPage(4)}>4</Pagination.Item>  
    </Pagination>  
  </div>  
}
```

Pagination

When displaying the data, use `slice` to only show the items on the current page!

```
function SomeBigData() {  
  const [page, setPage] = useState(1)  
  return <div>  
    {  
      bigData.slice((page - 1) * 16, page * 16).map(name => <p>{name}</p>)  
    }  
    {/* Display Pagination Items here! */}  
  </div>  
}
```

Manging State

Notice how the state disappears when flipping through the pages? Why is this?

How do we talk back to our parent? How do siblings talk to each other?

- Using callbacks
- Using `useContext`
- Using `sessionStorage` and `localStorage`
- Using cookies and an API (next time!)
- Using third-party libraries like (on your own!)...Redux, Recoil, MobX, XState

Passing Callbacks

The original way to do child-to-parent communication.

```
function TodoList() {
  const [items, setItems] = useState([
    { id: 1, text: 'Buy milk' },
    { id: 2, text: 'Walk dog' },
  ]);

  const removeItem = (id) => {
    setItems(prev => prev.filter(it => it.id !== id)); // update
  };

  return (
    <div>
      {items.map(it => (
        <TodoItem key={it.id} id={it.id} text={it.text} remove={removeItem} />
      ))}
    </div>
  );
}
```

```
function TodoItem({ id, text, remove }) {
  return (
    <div>
      {text}
      <button onClick={() => remove(id)}>Remove</button>
    </div>
  );
}
```

useContext Hook

A useful hook for managing state across web apps with large component hierarchies.

Motivation:

How can we effectively manage state for web apps with large component hierarchies?

- Spotify
 - SpotifyLandingPage
 - RecentSearches
 - AuthorCard
 - AuthorImage
 - AuthorName

Passing props down and down and down is known as "props-drilling" and is considered bad practice.

useContext Hook

Three steps to using context.

1. Create and export a context.
2. Provide the context with some value.
3. Use the context in a child component.

Often used in combination with useState.

useContext Hook

A context must be created and exported. Think of this like creating a data type!

```
import { createContext } from "react";  
  
// Create the "type" of shared data  
const MyDataContext = createContext(/* optional defaultValue */);  
  
export default MyDataContext;
```

useContext Hook

A context must be provided to child component(s).

```
function ParentComponent() {  
  const [data, setData] = useState(["Apples", "Oranges", "Bananas", "Grapes"]);  
  
  return (  
    <MyDataContext.Provider value={data}>  
      <SomeChildComponent />  
      <SomeOtherChildComponent />  
    </MyDataContext.Provider>  
  );  
}
```


useContext Hook

The context can be used by any of the child, grandchild, great-grandchild, etc. component(s).

```
function SomeChildComponent() {  
  const data = useContext(MyDataContext); // ← read the nearest Provider's value  
  
  return (  
    <>  
      {/* Do something interesting with data here */}  
      <ul>{data.map(item => <li key={item}>{item}</li>)}</ul>  
    </>  
  );  
}
```

ICA 8

BadgerMart

Welcome to the BadgerMart!

Available Groceries

apples: 2
bananas: 10
coconuts: 4

Shopping Cart

apples: 8
bananas: 0
coconuts: 0

Add apples	Add bananas	Add coconuts
------------	-------------	--------------

Shopping Cart

apples: 0
bananas: 6
coconuts: 0

Add apples	Add bananas	Add coconuts
------------	-------------	--------------

Shopping Cart

apples: 0
bananas: 0
coconuts: 0

Add apples	Add bananas	Add coconuts
------------	-------------	--------------

ICA 8

BadgerMart

```
BadgerMart
├─ <h1>Welcome to the BadgerMart!</h1>
├─ GroceryContext.Provider    (shares: [groceries, setGroceries])
│   ├── GroceryAisle         (reads groceries)
│   ├── ShoppingCart #1      (reads & updates groceries; has its own myCart)
│   ├── ShoppingCart #2      (reads & updates groceries; has its own myCart)
│   └─ ShoppingCart #3       (reads & updates groceries; has its own myCart)
```

ICA 8

BadgerMart

This mini-app demonstrates shared state with useContext:

- There is a global grocery inventory (how many apples/bananas/coconuts are left).
- Multiple components can read that inventory.
- Multiple components can update that inventory.
- Each ShoppingCart also has its own local cart state (how many items this cart has added).

So we're seeing both:

- Shared state via Context (groceries)
- Component-local state via useState (myCart)

ICA 8

Data-flow illustration (who owns what)

```
BadgerMart owns groceries state
  groceries + setGroceries
    ↓ (via Provider)
  GroceryAisle, Cart1, Cart2, Cart3 can access it
```

- A) GroceryContext.jsx: defines a **Context** object.
- B) BadgerMart.jsx (the root + shared state owner): defines a **State** object for the shared inventory, wraps components to access this **State**.
- C) GroceryAisle.jsx (reads shared state): uses **useContext** to access the shared inventory (**State**).
- D) ShoppingCart.jsx: defines a **State** object the local inventory of Cart, also updates the shared inventory